Clasificatori Bayes

Benchea Matei, grupa 252

1. Introducere

Proiectul de față implementează în Python doi clasificatori bayesieni (unul naiv și unul nenaiv) în vederea clasificării persoanelor cu diabet, oferind un verdict cu privire la prezența sau absența bolii (acestea fiind clasele noastre). Datele tabulare de antrenament și testare sunt publice, preluate de pe Kaggle, și conțin verdictul final, respectiv 21 de indicatori medicali relevanți în deciderea acestuia.

2. Clasificatorii Bayes

Un clasificator Bayes se ocupă cu următoarea problemă: *Dat fiind un obiect x cu caracteristicile $(x_1, x_2, \ldots, x_m)$ și o colecție de clase $C = (c_1, c_2, \ldots, c_k)$, care este probabilitatea ca x să aparțină unei clase c_i?* (în cazul nostru, x_j ar reprezenta una din cele 21 de date medicale din tabel, iar c_1=*"persoana are diabet"* și c_2=*"persoana nu are diabet"*). Un clasificator își bazează răspunsul pe Teorema lui Bayes: $P(c_i|x) = \frac{P(x|c_i) \cdot P(c_i)}{P(x)}$. Probabilitatea $P(c_i)$ (probabilitatea *prior*) spune cât de frecventă este clasa înainte să vedem caracteristicile, iar $P(x|c_i)$ indică cât de tipice sunt acestea pentru acea clasă. Clasificatorul calculează fiecare probabilitate $P(c_i|x)$, iar verdictul său este că x aparține clasei pentru care $P(c_i|x)$ este maxim (de notat aici că întreg calculul poate fi scalat cu $P(x)$, deoarece aceasta rămâne constantă pentru fiecare clasă). Modelul este *generativ*: din datele de antrenare învățăm, prin numărări, cum arată fiecare clasă, apoi inversăm raționamentul cu formula lui Bayes.

2.1 Clasificatorul naiv

Clasificatorul naiv pornește de la premisa că, dacă cunoaștem o clasă și o caracteristică, aceasta nu influențează cu nimic probabilitățile celorlalte caracteristici (i.e. sunt independente). Vom avea așadar că $P(x|c_i) = \prod_{j=1}^{m} P(x_j|c_i)$. Notăm cu n numărul de exemple de antrenare, N_{c_i} numărul celor din clasa c_i și N_{1,c_i} numărul celor din clasa c_i cu $x_j = 1$. Estimăm prin frecvențe relative $P(c_i) = N_{c_i}/n$ și $P(x_j{=}1|c_i) \approx N_{1,c_i}/N_{c_i}$. Pentru a putea calcula acum $P(c_i|x) \cdot P(x)$ ar trebui doar să folosim formula lui Bayes, dar riscăm să avem problema de underflow, așa că vom calcula produsul logaritmat: $\ln(P(c_i|x) \cdot P(x)) = \ln(P(c_i)) + \sum_{j=1}^{m} \ln(P(x_j|c_i))$. Cum ln este o funcție strict crescătoare, stabilirea maximului se poate face doar prin comparare.

2.2 Clasificatorul nenaiv

În realitate, în domeniul medical lucrăm adesea cu date care nu sunt independente (de exemplu, tensiunea arterială ridicată și colesterolul). Dacă am aplica regula lanțului fără nicio restricție pentru a modela aceste dependențe, am avea $P(x|c_i) = \prod_{j=1}^{m} P(x_j|c_i, x_1, \ldots, x_{j-1})$. Însă ultimii termeni ar depinde de mult prea multe variabile și ar necesita 2^{j-1} configurații, fiind imposibil de estimat corect din datele de antrenament limitate.

Ca să rezolvăm această problemă, facem un compromis: lăsăm fiecare caracteristică x_j să depindă, pe lângă clasa c_i, doar de cel mult k caracteristici anterioare (pe care le vom numi *dependențe*). O notă importantă aici este că alegem să luăm în calcul doar dependențele anterioare pentru ca graful acestora să rămână implicit aciclic (menționez că putem permite cicluri în graful nostru, dar atunci calculul probabilităților ar deveni mult mai complex, necesitând algoritmi avansați de inferență aproximativă), notate cu $\text{dep}(j) \subseteq \{1, \ldots, j-1\}$. Formula devine astfel:

$$P(x|c_i) = \prod_{j=1}^{m} P\big(x_j|c_i,\, x_{\text{dep}(j)}\big), \qquad P(x_j{=}1|c_i, u) \approx \frac{N_{1,c_i,u}}{N_{c_i,u}},$$

unde $u \in \{0,1\}^{|\text{dep}(j)|}$ reprezintă configurația (masca) de valori ale dependențelor lui j.

Pentru a alege dependențele unei caracteristici j, calculăm coeficientul de corelație Pearson și selectăm primele cel mult k caracteristici $l < j$ cele mai puternic corelate cu j (cu condiția ca $|\rho_{jl}| \geq 0.01$). De observat aici că dacă alegem $k = 0$ obținem exact modelul clasificatorului naiv.

2.3 Netezirea Laplace

Uneori putem întâmpina următoarea problemă: când calculăm din frecvențe probabilitățile condiționate $P(x_j = 0/1|c_i, u)$, este posibil ca pentru o anumită combinație să nu avem niciun exemplu în setul de antrenament ($N_{1,c_i,u} = 0$). Acest lucru ar face probabilitatea egală cu 0, anulând complet scorul clasei respective.

Pentru a evita acest blocaj, folosim **Netezirea Laplace**, care transformă un raport de forma x/y în $(x{+}\alpha)/(y{+}\text{cnt}{\cdot}\alpha)$, unde cnt reprezintă numărul de valori posibile ale caracteristicii (în cazul nostru va fi mereu 2, vezi secțiunea de implementare). Vom avea astfel: $P(x_j{=}1|c_i, u) = \frac{N_{1,c_i,u}+\alpha}{N_{c_i,u}+2\alpha}$

Astfel, probabilitățile rămân mereu strict în intervalul $(0, 1)$. Pentru o configurație care nu apare deloc în date ($N_{c_i,u} = 0$), probabilitatea estimată va fi neutră (1/2), iar pentru grupurile cu foarte multe exemple, influența lui α devine neglijabilă.

3. Detalii de implementare

Procesarea datelor: Setul de date (găsit în **data/** în format CSV) este prelucrat în cadrul modulului **src/prep.py**. În cadrul modulului parsăm fișierul CSV cu ajutorul bibliotecii *pandas* și îl împărțim aleatoriu, pe baza unui `seed`, în date de antrenament și date de testare (în mod implicit, 20% pentru test), izolând clasa țintă de restul caracteristicilor. Deoarece anumite coloane au valori numerice nebinare (e.g. vârsta sau venitul), datele trec printr-un proces de binarizare în funcția `to_bin` (le transformăm în 0 dacă sunt sub un prag și 1 altfel). Pragul de separare este dat de mediana calculată exclusiv pe setul de antrenament, aplicată apoi ca filtru atât peste datele de antrenament, cât și peste cele de testare.

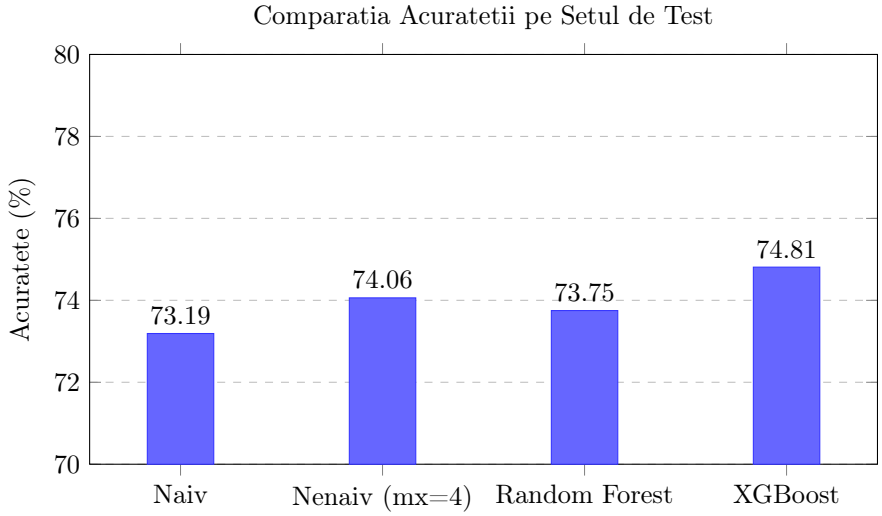
Clasificatorul naiv: Implementat în fișierul **src/naive_bayes.py**, definește clasa `naive_bayes` care gestionează logica modelului de bază. În etapa de antrenare (metoda `fit`), algoritmul determină mai întâi probabilitățile *prior* ale claselor, după care construiește o structură (`ps[i][j][nr]`$= P(x_{nr} = i|c = j)$) pentru a precalcula și stoca probabilitățile condiționate ale fiecărei caracteristici. Aici folosim Netezirea Laplace menționată anterior, prin intermediul parametrului `alf` (setat implicit la 1.0). În faza de predicție, pentru a evita fenomenul de *underflow* care apare la înmulțirea probabilităților mici, metoda `calc_logs` transformă produsele în sume de logaritmi, decizia finală pentru fiecare pacient fiind luată prin simpla comparare a acestor scoruri logaritmate.

Clasificatorul nenaiv: Implementat în fișierul **src/non_naive_bayes.py**, extinde modelul de bază prin renunțarea la ipoteza de independență totală. În faza de preprocesare a antrenării, modelul calculează matricea de corelație a setului de date pentru a asocia fiecărei caracteristici cel mult `mx_par` variabile anterioare (dependențe), selectate în ordinea descrescătoare a corelației absolute (folosind `np.corrcoef`). Deoarece calculul clasic prin parcurgerea independentă a tuturor celor 2^{mx_par} configurații posibile ale dependențelor ar fi fost extrem de ineficient, implementarea recurge la o tehnică avansată de vectorizare: stările binare ale dependențelor pentru fiecare pacient sunt comprimate într-o mască de biți. Acest identificator unic combină informația despre clasă și configurație, permițând extragerea frecvențelor dintr-o singură trecere peste întregul set de date cu ajutorul funcției `np.bincount`. Probabilitățile condiționate obținute în urma numărătorii și aplicării Netezirii Laplace sunt stocate într-o structură de dicționare, permițând ca, la momentul predicției, calculul scorurilor logaritmate să se facă prin accesări rapide, direct pe baza măștii reconstruite.

Diverse: Din **main.py** putem rula ambele modele pentru a le compara acuratețea pe datele de antrenament, respectiv de testare. Când rulăm din linia de comandă, putem alege inclusiv `seed`-ul, numărul maxim de dependențe pentru modelul nenaiv și valoarea lui α pentru Netezirea Laplace. În plus, în proiect mai găsim și **exp.py**, program folosit pentru generarea datelor din graficele secțiunii următoare.

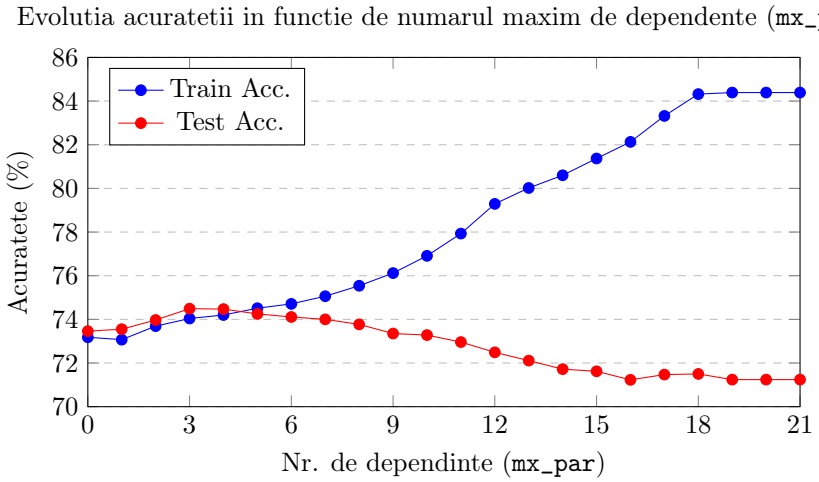
4. Rezultate

În continuare am testat acuratețea modelelor pe datele de testare în comparație cu modele de top din domeniu: Random Forest și XGBoost. Pentru corectitudinea experimentului, modelele Bayes au primit datele binarizate, în timp ce Random Forest și XGBoost le-au primit în formatul original. Datele din următorul grafic au fost obținute după 50 de iterații de antrenare și testare (fiecare cu un `seed` aleator din $[1, 10^6]$), calculând ulterior media acuratețelor pentru fiecare model.



În continuare, pentru a evalua impactul dependențelor dintre caracteristici, am rulat clasificatorul pentru toate valorile posibile ale parametrului `mx_par` de la 0 (echivalent cu clasificatorul naiv) până la 21. Am măsurat acuratețea

atât pe setul de antrenament (Train Acc.), cât și pe cel de testare (Test Acc.) pe parcursul a 5 iterații (fiecare cu un `seed` aleator din $[1, 10^6]$), după care am calculat media acurateților:



Din graficul de mai sus putem trage două concluzii importante:

- **Numărul optim de dependențe permise**: Observăm că pentru `mx_par=3, 4` obținem cea mai bună acuratețe pe datele de testare. Permițând corelarea dintre indicatorii medicali, modelul reușește să capteze tipare mai apropiate de realitate.
- **Supraînvățarea (Overfitting)**: Pe măsură ce `mx_par` crește, acuratețea pe datele de testare își atinge maximul la valoarea 3, iar apoi scade, în timp ce acuratețea pe datele de antrenament continuă să crească foarte rapid. Acest fenomen reprezintă *overfitting-ul*: pentru un număr mare de dependențe permise, spațiul configurațiilor posibile devine imens, astfel că fiecare configurație ajunge să fie reprezentată de foarte puține exemple. În acest punct, modelul începe să memoreze particularitățile setului de antrenament în loc să generalizeze.